- understanding prob. definitions
- proving alg. correctness
- describing algorithm efficiency
  scalability

"mergesort is $O(n \log n)$" ???

"insertion sort is $O(n^2)$" . . .

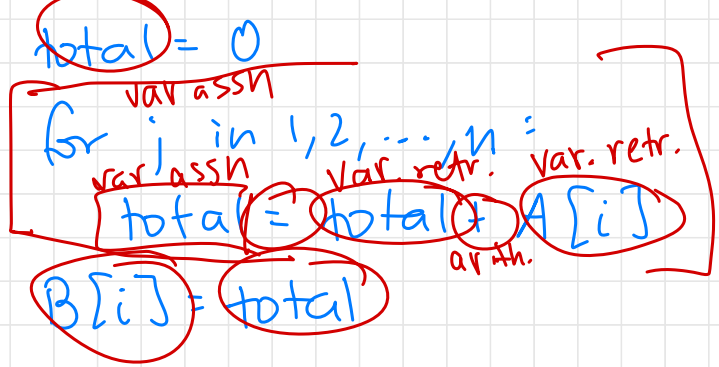
Runtime = # of primitive operations an algorithm takes on input of size n

basic arithmetic
+, −, *, ~
variable assignment
variable retrieval
boolean operations = ≠

Algorithm1 (int array A of length n):

initialize int B of length n

for i in 1, 2, ..., n:



"inner loop"

prim ops done by one execution of inner loop?

5

let $f(n)$ = # primitive operations used by Alg 1 on input size n

what $f(n)$? some function of n...

$f(n) = 6n + 5n^2$

$= 4n + 5n^2$

$= 4n + 5n^2 + 1$

$6n^2$ over $10n^2$

ops per outer loop?

$1 + 1 + 1 + 1 + 5n$

$= 4 + 5n$

total outer loop:

$n(4 + 5n) = 4n + 5n^2$

pros

it's true!

it's exact

cons

too exact

- diff impl. use diff. # prim ops
- comparison

"Alg 1 is $O(n^2)$"

"Alg 1 takes some $f(n)$ # of prim. ops. on an input of size n, where $f(n)$ is $O(n^2)$"

Asymptotic Bounds

big O, Ω, Θ

Let $f(n)$ and $g(n)$ be functions

$f(n) = O(g(n))$ if $f(n)$ is asymptotically upper bounded by $g(n)$.

$f(n)$ is $O(g(n))$ if $f(n)$ is upper bounded by a constant multiple of $g(n)$ for sufficiently large n.

① ② ③

ex $f(n) = 10n^2 + 3n + 1000$

$g(n) = n^2 \cdot 5000$

is $f(n) = O(g(n))$?

are ①, ③ T?

are ①, ②, ③ T?

Properties:

$$a^{n+m} = a^n a^m$$

$$a^{nm} = (a^n)^m$$

$$(ab)^n = a^n b^n$$

Properties of Logarithms:

$$\log_b xy = \log_b x + \log_b y$$

$$\log_b \frac{x}{y} = \log_b x - \log_b y$$

$$\log_b x^y = y \log_b x$$

$$\log_b x = \frac{\log_c x}{\log_c b}$$